CineApp DSM

Documentación Técnica del Proyecto

Plataforma de Video on Demand (VOD)

Juan Lacosta y Yoel Agirretxe

Repositorio GitHub: https://github.com/jlacostaarpide/Proyecto-Webapp-Dsm2026

Despliegue Firebase: https://webapp-react-dsm2026.web.app/

Arquitectura y Características de React

El proyecto utiliza una arquitectura basada en componentes funcionales, aprovechando las capacidades modernas de React para gestionar el estado y los efectos secundarios.

1. Gestión del Estado Global (Context API)

Se ha implementado `AuthContext` para gestionar el estado de autenticación en toda la aplicación. Este enfoque permite centralizar la información del usuario (como login, username, idToken y userId) evitando el problema conocido como *prop drilling*, que ocurre cuando los datos deben pasarse manualmente a través de múltiples niveles de componentes.
El contexto se define como un proveedor global que envuelve toda la aplicación dentro de `App.jsx`, permitiendo que cualquier componente hijo pueda acceder a estos datos sin necesidad de recibirlos explícitamente como props:

```
<AuthContext.Provider value={{ login, username, idToken, userId, onLogout:
handleLogout }}>
  <Header />
  <main>...</main>
  <Footer />
</AuthContext.Provider>
```

Esto introduce una arquitectura desacoplada, donde los componentes pueden consumir únicamente la información que necesitan mediante el hook `useContext(AuthContext)`.

Los componentes que hacen uso directo de este contexto son:

- `Header.jsx`: para mostrar información del usuario y gestionar logout
- `MovieDetail.jsx`: para permitir votar, comentar o añadir favoritos
- `Favorites.jsx`: para acceder a los datos del usuario autenticado

Además, el uso del contexto permite implementar funcionalidades clave como:

- Persistencia de sesión
- Control de acceso a rutas protegidas
- Personalización de la interfaz según el usuario

2. Hooks de React

El proyecto hace un uso intensivo de los hooks, que permiten gestionar estado y lógica en componentes funcionales sin necesidad de clases

useState

Se utiliza para gestionar estados locales en prácticamente todos los componentes. Algunos ejemplos concretos del proyecto:

- `searchQuery` y `categoriaActiva` en `Home.jsx`
- Estados de formulario en `Login.jsx`, `Register.jsx` y `Contact.jsx`
- Estados complejos en `MovieDetail.jsx`:
    - `averageRating`
    - `totalRatings`
    - `comments`
    - `isFavorite`

- `hasVoted`
- `hasCommented`

Esto permite que la interfaz sea completamente dinámica y reactiva.

useEffect:

Se utiliza para gestionar efectos secundarios, especialmente llamadas a APIs externas (Firebase).

Casos concretos del proyecto:

- Carga de ratings, comentarios y favoritos al montar `MovieDetail.jsx`
- Obtención de favoritos en `Favorites.jsx`
- Restauración de sesión desde `localStorage` en `App.jsx`

Esto permite sincronizar el estado de React con datos externos.

useContext:

Consume el `AuthContext` para obtener `login`, `username`, `idToken` y `userId` sin necesidad de props.

useParams:

Extrae el parámetro dinámico `:id` de la URL en `MovieDetail.jsx` para identificar qué película mostrar.

useNavigate:

Permite redirecciones programáticas:

- Redirigir a `/login` si el usuario intenta acceder sin autenticación
- Volver a la home tras logout
- Navegar tras acciones exitosas

3. Props y flujo de datos entre componentes

En React, los datos fluyen de componentes padres a hijos mediante **props**. En este proyecto, `App.jsx` actúa como componente raíz y distribuye los datos necesarios:

- `Home` recibe `peliculas` (el array completo) para renderizar el catálogo y el carrusel.
- `MovieDetail` recibe `peliculas` para localizar la película activa por `id` y calcular las recomendaciones.
- `Favorites` recibe `peliculas` para cruzarlos con los IDs favoritos del usuario guardados en Firebase.
- `Login` recibe `onLogin`, una función callback que, al ejecutarse, actualiza el estado global en `App.jsx` (patrón *lifting state up*).

Este patrón permite que los componentes hijos notifiquen cambios al padre sin acoplarse directamente.

El proyecto sigue el modelo clásico de React de flujo de datos unidireccional, donde los datos se transmiten desde componentes padre a hijos mediante props.

`App.jsx` actúa como fuente principal de datos, distribuyendo el array de películas a los distintos componentes:

- `Home.jsx`
  - Renderiza el catálogo completo
  - Aplica filtros y búsquedas
- `MovieDetail.jsx`
  - Busca una película concreta por id
  - Genera recomendaciones
- `Favorites.jsx`
  - Cruza IDs de Firebase con el array completo

Además, se utiliza el patrón de lifting state up, donde los componentes hijos comunican cambios al padre mediante callbacks.

Ejemplo:

- `Login.jsx` recibe `onLogin`
- Al ejecutarse, actualiza el estado global en `App.jsx`

Esto permite mantener una arquitectura limpia y escalable.

4. Renderizado condicional

El proyecto hace un uso intensivo del renderizado condicional para adaptar la interfaz al estado del usuario:

- **Header.jsx**: Muestra "Login / Registro" si el usuario no está autenticado, o el nombre de usuario y el botón de logout si lo está.
- **MovieDetail.jsx**: Muestra el formulario de comentarios solo si el usuario está logueado y aún no ha comentado. Si ya comentó, muestra un aviso. Si no ha iniciado sesión, muestra un enlace al login.
- **MovieDetail.jsx**: El botón de favoritos cambia su texto, color e icono según el estado de `isFavorite`.
- **Favorites.jsx**: Redirige automáticamente a `/login` si el usuario no está autenticado, actuando como ruta protegida.
- **Login.jsx / Register.jsx**: Muestran una pantalla de éxito en lugar del formulario cuando la operación se completa correctamente.

5. Componentes controlados (Controlled Components)

Los formularios del proyecto utilizan el patrón de componentes controlados, en el cual el valor de cada `<input>` o `<textarea>` está enlazado a una variable de estado mediante la propiedad `value`, y se actualiza a través del evento `onChange`. De esta forma, React mantiene el control total del formulario, actuando como fuente única de verdad.

Este enfoque permite gestionar de forma centralizada los datos introducidos por el usuario, facilitando la validación, el control del flujo de la interfaz y la integración con otras funcionalidades de la aplicación.

Ejemplo en `Login.jsx`, donde un único `handleChange` gestiona todos los campos usando el atributo `name`:

```
const handleChange = (e) => {
  setFormData({ ...formData, [e.target.name]: e.target.value });
};

<Form.Control name="email"    value={formData.email}    onChange={handleChange} />
<Form.Control name="password" value={formData.password} onChange={handleChange} />
```

Este patrón permite evitar múltiples funciones de actualización y facilita la escalabilidad del formulario.

En `Register.jsx`, este mismo enfoque se combina con validaciones en cliente, como la comprobación de que las contraseñas coinciden o cumplen una longitud mínima antes de enviarlas a Firebase.

En `Contact.jsx`, se utiliza igualmente un estado controlado para gestionar los campos del formulario (nombre, email y mensaje), permitiendo mostrar una confirmación tras el envío sin necesidad de recargar la página.

Por último, en `MovieDetail.jsx`, el formulario de comentarios también sigue este patrón, con la particularidad de que su renderizado depende del estado del usuario: solo se muestra si está autenticado y no ha comentado previamente.

6. Enrutamiento (React Router v7)

Se utiliza un sistema de rutas dinámico para separar las vistas de la aplicación:

Ruta	Componente	Tipo
/	Home	Pública
/login	Login	Pública
/register	Register	Pública
/movie/:id	MovieDetail	Dinámica
/favorites	Favorites	Protegida
/contact	Contact	Pública
/legal	LegalNotice	Pública
*	ErrorPage	Comodín (404)

7. Persistencia de Datos (Google Firebase)

Se ha integrado toda la persistencia en la nube mediante la **API REST de Firebase** usando **Axios**:

- **Firebase Authentication**: Gestión real de usuarios con el Identity Toolkit de Google.
- **Firebase Realtime Database**: Almacenamiento en tiempo real de favoritos, valoraciones y comentarios.
- **Seguridad**: Las peticiones a datos privados incluyen el `idToken` en la URL (`?auth={idToken}`) para autenticación.
- **Persistencia de sesión**: Los tokens (`idToken`, `userId`, `email`) se guardan en `localStorage` para que la sesión sobreviva a recargas de página.

Descripción de Componentes

Componentes de Interfaz (UI)

Header.jsx

Barra de navegación fija en la parte superior de todas las páginas. Utiliza `useContext(AuthContext)` para leer el estado de autenticación y renderiza contenido diferente según el caso:

- **Usuario no autenticado**: Muestra los enlaces "Login" y "Registro".
- **Usuario autenticado**: Muestra el enlace "Mis Favoritos", el nombre del usuario y un botón de logout.

Al hacer logout llama a `onLogout()` del contexto (que limpia el estado en `App.jsx` y el `localStorage`) y redirige al inicio con `useNavigate`.

Footer.jsx

Pie de página estático presente en todas las vistas. Contiene el copyright de la aplicación y enlaces a las páginas de Aviso Legal y Contacto.

Peliculas.jsx

Componente contenedor que recibe un array `peliculas` como prop y lo renderiza en una cuadrícula responsive con el sistema de grid de React Bootstrap (`Row` y `Col`). Configura el número de columnas según el tamaño de pantalla: 1 en móvil, 2 en tablet, 4 en escritorio.

Pelicula.jsx

Tarjeta individual de película. Recibe los datos de una película como props y muestra: imagen, título, categoría, descripción recortada y botón de acceso a la ficha. Al montarse ejecuta un `useEffect` que consulta Firebase para obtener en tiempo real la nota media y el número de comentarios, mostrándolos como badges dinámicos.

Páginas (Vistas principales)

Home.jsx

Página principal y punto de entrada al catálogo. Recibe el array `peliculas` como prop y gestiona dos estados locales:

- `categoriaActiva` (useState): Categoría seleccionada actualmente.
- `searchQuery` (useState): Texto introducido en el buscador.

El array de películas filtradas se calcula directamente en el render combinando ambos filtros:

```
const peliculasFiltradas = peliculas.filter(p => {
  const matchesCategory = categoriaActiva === "Todas" || p.categoria ===
categoriaActiva;
  const matchesSearch = p.titulo.toLowerCase().includes(searchQuery.toLowerCase());
  return matchesCategory && matchesSearch;
});
```

Las categorías del filtro se generan dinámicamente con `Set` para evitar duplicados. Incluye un **carrusel** con las 3 primeras películas y un estado vacío con botón para resetear los filtros.

MovieDetail.jsx

El componente más complejo del proyecto. Recibe `peliculas` como prop y usa `useParams` para obtener el `:id` de la URL. Gestiona 9 estados locales independientes y realiza 3 llamadas a Firebase en el mismo `useEffect`:

- **Valoraciones**: Sistema interactivo de 5 estrellas. `hoverRating` controla el efecto visual previo al clic. `averageRating` y `totalRatings` se calculan desde Firebase. `hasVoted` (por `userId`) bloquea votos duplicados.
- **Favoritos**: Consulta Firebase al cargar para saber si la película ya es favorita (`isFavorite`). El botón toggle ejecuta `PUT true` para añadir o `DELETE` para quitar.
- **Comentarios**: Carga los comentarios desde Firebase y comprueba con `hasCommented` si el usuario ya escribió uno. Los nuevos comentarios se publican con `POST`. El hilo se muestra en orden cronológico inverso con `.slice().reverse()`.
- **Recomendaciones**: Array derivado calculado filtrando `peliculas` por la misma categoría y excluyendo la película actual, mostrando hasta 4 sugerencias.

Login.jsx

Formulario de inicio de sesión con componente controlado. Un único `handleChange` usa `e.target.name` como clave dinámica para gestionar todos los campos. Al enviar realiza un `POST` al endpoint `signInWithPassword` de Firebase Identity Toolkit. Si la respuesta es correcta, llama a la prop `onLogin` con el `idToken` y `localId`, actualizando el estado global en `App.jsx`. Distingue mensajes de error específicos: credenciales incorrectas, cuenta deshabilitada o demasiados intentos.

Register.jsx

Formulario de registro de nuevos usuarios. Realiza validaciones en cliente: las contraseñas deben coincidir y tener al menos 6 caracteres. Si la validación pasa, hace un `POST` al endpoint `signUp` de Firebase. Detecta el error `EMAIL_EXISTS` para informar al usuario. Al completarse con éxito muestra una pantalla de confirmación con enlace al login.

Favorites.jsx

Página protegida que muestra las películas favoritas del usuario activo. Usa `useContext(AuthContext)` para obtener `login`, `userId` e `idToken`. En el `useEffect`, si el usuario no está autenticado, redirige a `/login` con `useNavigate`. Si lo está, realiza un `GET` a Firebase a `/favorites/{userId}`, cruza los IDs con el array `peliculas` recibido por prop y actualiza el estado `favoritePeliculas`. Muestra el grid de películas o un estado vacío si el usuario aún no tiene favoritos.

Contact.jsx

Página estática con formulario de contacto (nombre, email, mensaje). Gestiona los campos como componente controlado con `useState`. Al enviar muestra una pantalla de confirmación. Cumple con los requisitos de información de la aplicación.

LegalNotice.jsx

Página estática con tres secciones de información legal: Aviso Legal (cumplimiento LSSICE), Política de Privacidad (cumplimiento GDPR) y Política de Cookies. Incluye un aviso de uso educativo.

ErrorPage.jsx

Página de error 404. Se activa automáticamente con la ruta comodín `path="*"` en `App.jsx`, capturando cualquier URL no reconocida. Muestra un mensaje de error claro y un enlace para volver al inicio.

Estructura de Datos en Firebase

La aplicación utiliza Firebase Realtime Database como backend en la nube. Los datos se organizan en tres colecciones principales:

Valoraciones (/ratings)

```
{
  "ratings": {
    "{movieId}": {
      "{userId}": 4
    }
  }
}
```

Cada película tiene un nodo con el `userId` de cada votante como clave y su puntuación numérica (1-5) como valor. Esto garantiza un único voto por usuario por película. La media se calcula en cliente sumando todos los valores y dividiendo entre el total.

Comentarios (/comments)

```
{
  "comments": {
    "{movieId}": {
      "{autoId}": {
        "username": "nombre_usuario",
        "text": "Texto del comentario",
        "date": "17/3/2026"
      }
    }
  }
}
```

Firebase genera automáticamente un ID único para cada comentario al usar `POST`. Se verifica si el usuario ya comentó comparando su `username` con los comentarios existentes.

Favoritos (/favorites)

```
{
  "favorites": {
    "{userId}": {
      "{movieId}": true
    }
  }
}
```

Cada usuario tiene su propio nodo con los IDs de sus películas favoritas como claves y `true` como valor. Para añadir se usa `PUT true` y para eliminar `DELETE`. Las peticiones a `/favorites` requieren el `idToken` de Firebase como autenticación (`?auth={idToken}`).

Tecnologías Utilizadas

Tecnología	Versión	Uso en el proyecto
React	19.2.4	Biblioteca principal para la interfaz de usuario
Vite	8.0.0	Herramienta de construcción y servidor de desarrollo
React Router	7.13.1	Enrutamiento SPA, rutas dinámicas y navegación programática
React Bootstrap	2.10.10	Componentes UI responsive (Navbar, Carousel, Card, Alert...)
Bootstrap	5.3.8	Sistema de grid y estilos base
Axios	1.13.6	Cliente HTTP para comunicación con la API REST de Firebase
Firebase SDK	12.10.0	Configuración de proyecto y despliegue en Firebase Hosting
Firebase Realtime DB	(Cloud)	Base de datos en tiempo real para ratings, comentarios y favoritos
Firebase Auth	(Cloud)	Autenticación de usuarios mediante Identity Toolkit
CSS3	—	Estilos personalizados, efectos hover y diseño dark mode